

IRCTC-Style High Concurrency Backend Project

Phase 0: System Design & Planning

- 1 Define overall system architecture
- 2 Decide technologies: Go, PostgreSQL, Redis, Kafka
- 3 Design ER diagram and data flow
- 4 Define booking lifecycle and Tatkal behavior

Phase 1: Database Design

- 1 User authentication tables
- 2 Train, schedule, coach, seat modeling
- 3 Booking and booking item tables
- 4 Payment lifecycle handling
- 5 Tatkal quota management

Phase 2: Infrastructure Setup

- 1 PostgreSQL connection setup
- 2 Redis client setup
- 3 Configuration management
- 4 Project folder structure

Phase 3: Domain & Repository Layer

- 1 Create Go structs for DB tables
- 2 Repository layer for DB queries
- 3 Seat availability queries
- 4 Booking and payment DB operations

Phase 4: Seat Locking (Concurrency Core)

- 1 Redis seat locking using SETNX
- 2 TTL based auto-expiry
- 3 Prevent double booking
- 4 Handle lock release on failure

Phase 5: Booking APIs

- 1 Train search API
- 2 Seat availability API
- 3 Booking request API
- 4 Create PENDING booking
- 5 Return booking hold token

Phase 6: Kafka Event Driven Workflow

- 1 Kafka topics for booking and payment
- 2 Booking worker service
- 3 Payment worker service
- 4 Booking confirmation worker
- 5 Failure and retry handling

Phase 7: Tatkal Load Handling

- 1 Tatkal booking API
- 2 Redis rate limiting
- 3 Tatkal queue using Kafka
- 4 Quota enforcement
- 5 Fair request processing

Phase 8: Reliability & Observability

- 1 Booking expiry cleanup
- 2 Prometheus metrics
- 3 Structured logging
- 4 Load testing with k6
- 5 System stability validation